

Desenvolvimento de Sistemas

Aula 2 · Evolução do cadastro em Java Desktop

ArrayList, JOptionPane, validações simples e busca em memória

Onde estamos no semestre

A Aula 2 deixa de repetir o primeiro projeto e transforma a base em um pequeno cadastro.

TRILHA



Produto de hoje: cadastrar vários alunos, listar, buscar e validar entradas em um programa Java desktop simples.

Por que mudar a prática?

A segunda aula precisa avançar: de objetos soltos para fluxo de sistema.

MOTIVAÇÃO

Antes
objetos fixos no código
saída no Console
sem lista e sem busca

Agora
dados digitados pelo usuário
lista em memória
menu, busca e validação

Ganho pedagógico
os estudantes enxergam o
caminho dos dados em um
sistema pequeno

A complexidade aumenta sem perder o controle: cada novo recurso resolve um problema visível.

Objetivos da aula

Ao final, o estudante deve conseguir evoluir e explicar o próprio projeto.

OBJETIVOS

**Guardar vários objetos
ArrayList<Aluno> em
memória**

**Separar responsabilidades
Aluno, CadastroAlunos e
Principal**

**Interagir por janelas
menu com JOptionPane**

**Validar entradas
texto obrigatório e
semestre válido**

Critério de aprendizagem: explicar o fluxo entrada → objeto → lista → método → resposta.

Arquitetura mínima da prática

Ainda não é MVC completo, mas já existe separação de responsabilidades.

VISÃO GERAL



- Principal não deve guardar diretamente todas as regras.
- CadastroAlunos concentra operações sobre a lista.
- Aluno continua sendo a entidade simples do domínio.

Classe Aluno: pequeno ajuste

A entidade ganha getters para permitir busca sem expor os atributos.

DOMÍNIO

```
public class Aluno {  
    private String nome;  
    private int semestre;  
    private String curso;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public String gerarResumo() {  
        return nome + " - " + curso + " - " + semestre +  
" semestre";  
    }  
}
```

Ideia principal

Atributos continuam private.

Métodos controlam o acesso aos dados.

Isso prepara validações e organização por camadas.

ArrayList: cadastro em memória

Enquanto o programa está aberto, a lista guarda os objetos criados pelo usuário.

COLEÇÃO

```
import java.util.ArrayList;

public class CadastroAlunos {
    private ArrayList<Aluno> alunos = new ArrayList<>();

    public void adicionar(Aluno aluno) {
        alunos.add(aluno);
    }

    public int quantidade() {
        return alunos.size();
    }
}
```

Memória
Os dados somem ao fechar o programa.
Isso é esperado nesta etapa.
Persistência virá depois com JDBC.

Listar e buscar

A lista precisa ser percorrida para montar resultados ou encontrar um aluno.

PROCESSAMENTO

```
public Aluno buscarPorNome(String nome) {  
    for (Aluno aluno : alunos) {  
        if (aluno.getNome().equalsIgnoreCase(nome)) {  
            return aluno;  
        }  
    }  
    return null;  
}
```

- for-each percorre todos os objetos.
- equalsIgnoreCase compara textos ignorando maiúsculas/minúsculas.
- return null indica que nenhum aluno foi encontrado.

CEUB

EDUCAÇÃO SUPERIOR

Menu com JOptionPane

A interação desktop começa com caixas de diálogo simples.

INTERFACE

```
String menu = "1 - Cadastrar aluno\n"
             + "2 - Listar alunos\n"
             + "3 - Buscar aluno por nome\n"
             + "0 - Sair";

opcao = lerInteiro(menu, 0, 3);
```

Atenção
JOptionPane é recurso desktop simples.
Ainda não estamos criando telas completas.
O foco é entender o fluxo.

Menu → escolha → método chamado → resposta ao usuário

Validação inicial

O sistema deve reagir a entradas vazias ou incoerentes.

VALIDAÇÃO

Entrada	Validação	Resposta
Nome	não pode ficar vazio	avisar e pedir novamente
Curso	não pode ficar vazio	avisar e pedir novamente
Semestre	número entre 1 e 10	avisar se texto ou fora da faixa
Menu	número entre 0 e 3	não aceitar opção inexistente

Validação não é enfeite: é parte da regra de funcionamento do sistema.

Prática guiada da aula

A professora demonstra; depois a turma implementa e testa.

ROTEIRO

1

Abrir projeto anterior

2

Ajustar Aluno

3

Criar CadastroAlunos

4

Criar menu Principal

5

Testar cadastro/lista/busca

6

Validar entradas

Produto da prática: cadastro em memória com três classes e interação por JOptionPane.

Como testar o projeto

Teste guiado evita achar que o código funciona só porque compilou.

TESTES

Teste	O que verificar
Cadastrar 3 alunos	os dados aparecem na listagem
Listar com lista vazia	mensagem adequada
Buscar aluno existente	resumo correto
Buscar aluno inexistente	mensagem de não encontrado
Digite texto no semestre	sistema pede número novamente

Compilar é o primeiro passo. Testar cenários é parte do desenvolvimento.

Erros comuns na evolução

A aula também ensina a diagnosticar problemas frequentes.

DEPURAÇÃO

Sintoma	Causa provável	Ação
ArrayList em vermelho	import ausente	import java.util.ArrayList;
JOptionPane em vermelho	import ausente	import javax.swing.JOptionPane;
Busca nunca encontra	comparação com ==	usar equalsIgnoreCase
Erro ao digitar semestre	parse sem try/catch	usar validação com NumberFormatException

Atividade pontuada

Incrementar o cadastro em dupla.

ESTRELINHA

- Escolher um incremento: buscar por semestre, contar alunos por curso ou bloquear nome duplicado.
- Criar ou alterar método em CadastroAlunos.
- Adicionar uma nova opção no menu da classe Principal.
- Testar um caso com resultado e um caso sem resultado.
- Explicar o fluxo: entrada, objeto, lista, método e resposta.

Registro de participação

Vale pela evolução real do projeto e pela explicação do próprio código. Código sem compreensão não conta como entrega adequada.

Fechamento

O projeto agora já se comporta como um pequeno sistema.

SÍNTESE

Hoje
lista, menu, busca, validação

Próxima aula
telas desktop mais
organizadas

Depois
separação em camadas

Projeto final
sistema desktop persistente

Mensagem final: um sistema cresce por incrementos pequenos, testados e explicáveis.